03_clustering k-means and hierarchical

Anish

2026-05-08

1. Goal

Fit clusterers (k-means and hierarchical) on the PC scores from `02_PCA.Rmd`. Pick a `k` defensible under multiple criteria, validate stability via bootstrap, and save per-customer labels for `04_profiles.Rmd`.

2. Load and validate

```
pc       <- read_csv("../data/processed/customer_pc_scores.csv", show_col_types = FALSE)
customer <- read_csv("../data/processed/customer_features.csv",  show_col_types = FALSE)

stopifnot(
  "CustomerID" %in% names(pc),
  "CustomerID" %in% names(customer),
  !anyDuplicated(pc$CustomerID),
  setequal(pc$CustomerID, customer$CustomerID)
)

dim(pc)
```

```
## [1] 4334    4
```

```
glimpse(pc)
```

```
## Rows: 4,334
## Columns: 4
## $ CustomerID <dbl> 12346, 12347, 12348, 12349, 12350, 12352, 12353, 12354, 123~
## $ PC1        <dbl> -2.82716427, 3.34474112, 0.86447698, 0.42554416, -1.8673585~
## $ PC2        <dbl> -5.49318434, 0.22375290, -0.21402402, -2.26806776, -0.95978~
## $ PC3        <dbl> -0.37056405, -0.08693365, 0.79954652, -0.55067776, 0.420579~
```

```
Xpc <- pc |> select(starts_with("PC")) |> as.matrix()
rownames(Xpc) <- pc$CustomerID
n_pc <- ncol(Xpc)

sprintf("Clustering on %d PCs across %d customers", n_pc, nrow(Xpc))
```

```
## [1] "Clustering on 3 PCs across 4334 customers"
```

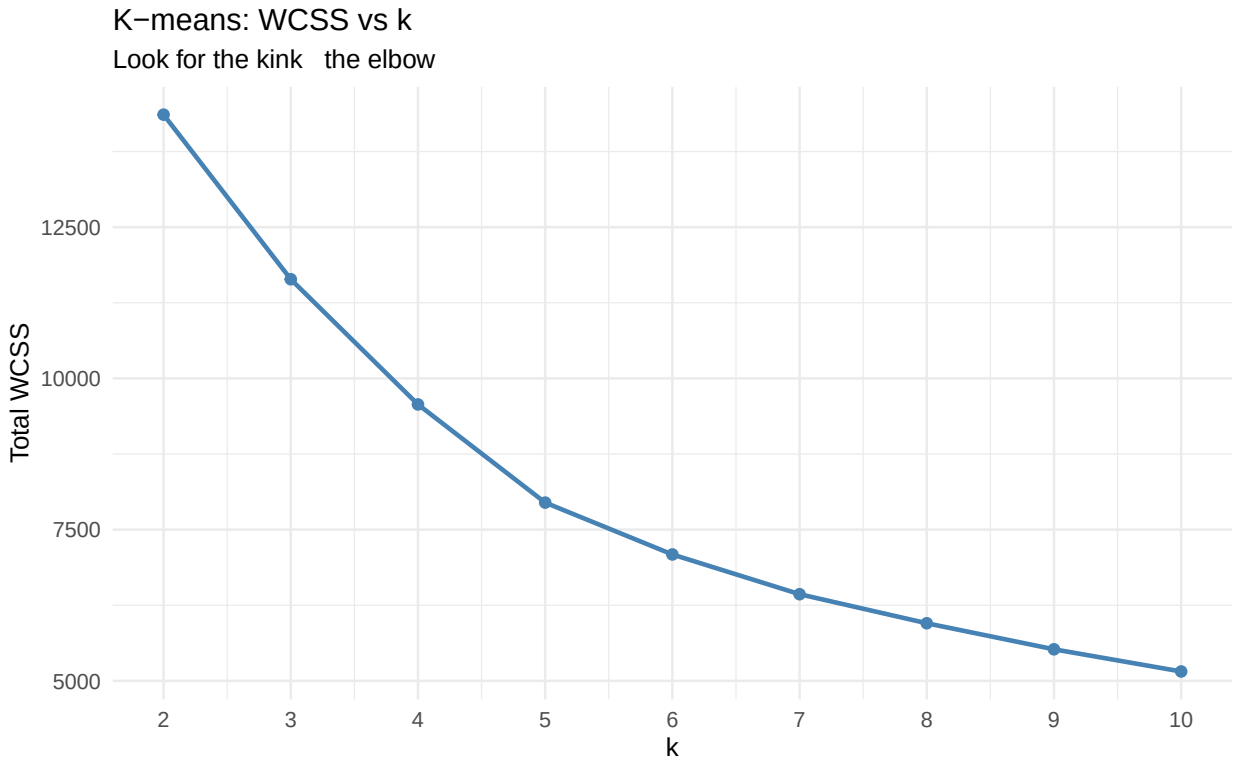
3. K-means choosing k

3a. Elbow (within-cluster SSE)

The elbow plot is a heuristic: WCSS strictly decreases as `k` grows, because more clusters can fit the data more tightly.

```
set.seed(1)
k_grid <- 2:10
wcss <- map_dbl(k_grid,
                \(k) kmeans(Xpc, centers = k, nstart = 25,
                            iter.max = 50)$tot.withinss)
elbow_df <- tibble(k = k_grid, wcss = wcss)

ggplot(elbow_df, aes(k, wcss)) +
  geom_line(colour = "steelblue", linewidth = 1) +
  geom_point(colour = "steelblue", size = 2) +
  scale_x_continuous(breaks = k_grid) +
  labs(title = "K-means: WCSS vs k",
       subtitle = "Look for the kink   the elbow",
       x = "k", y = "Total WCSS")
```

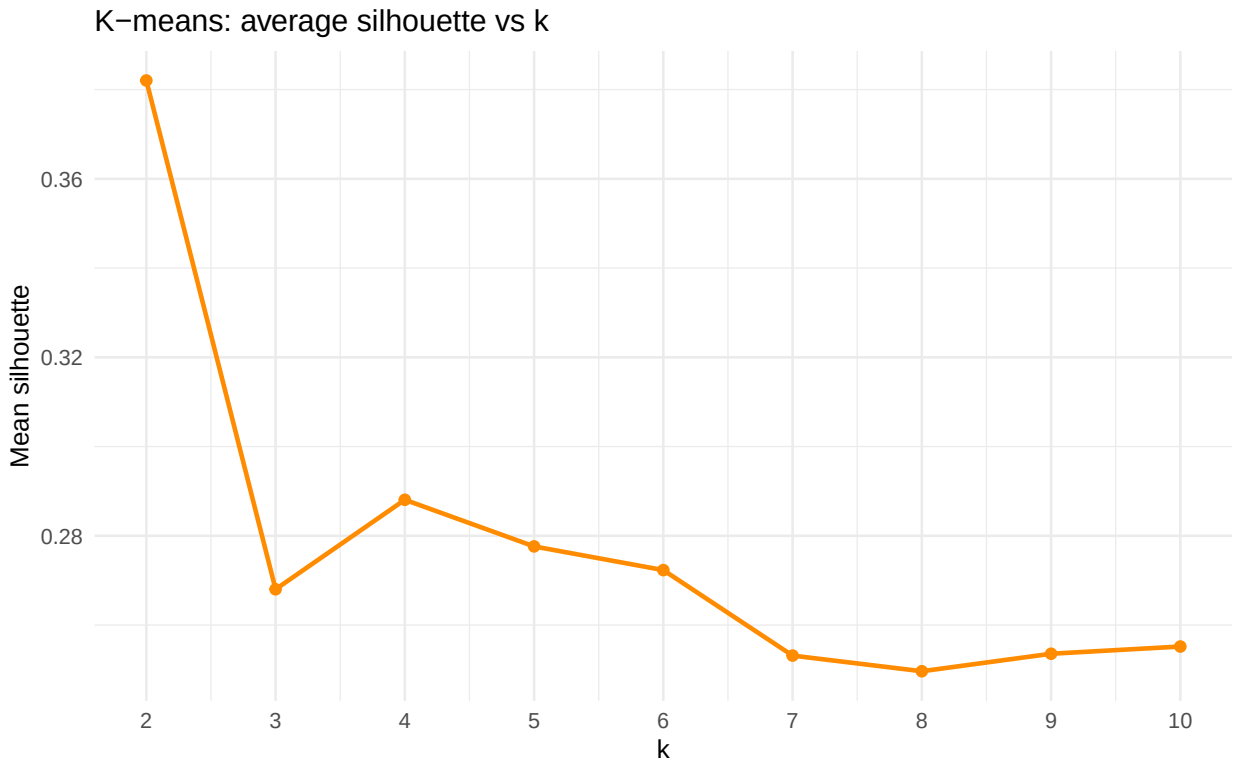


3b. Silhouette width

For each customer, silhouette width compares its average distance to others in its own cluster against its average distance to the *next-nearest cluster*. Mean silhouette ~ 0 means clusters overlap; ~ 1 means they're well separated.

```r
set.seed(1)
d <- dist(Xpc)
sil <- map_dbl(k_grid, \(k) {
  cl <- kmeans(Xpc, centers = k, nstart = 25, iter.max = 50)$cluster
  mean(silhouette(cl, d)[, "sil_width"])
})
sil_df <- tibble(k = k_grid, silhouette = sil)

ggplot(sil_df, aes(k, silhouette)) +
  geom_line(colour = "darkorange", linewidth = 1) +
  geom_point(colour = "darkorange", size = 2) +
  scale_x_continuous(breaks = k_grid) +
  labs(title = "K-means: average silhouette vs k",
       x = "k", y = "Mean silhouette")
```



3c. Gap statistic Tibshirani (2001)

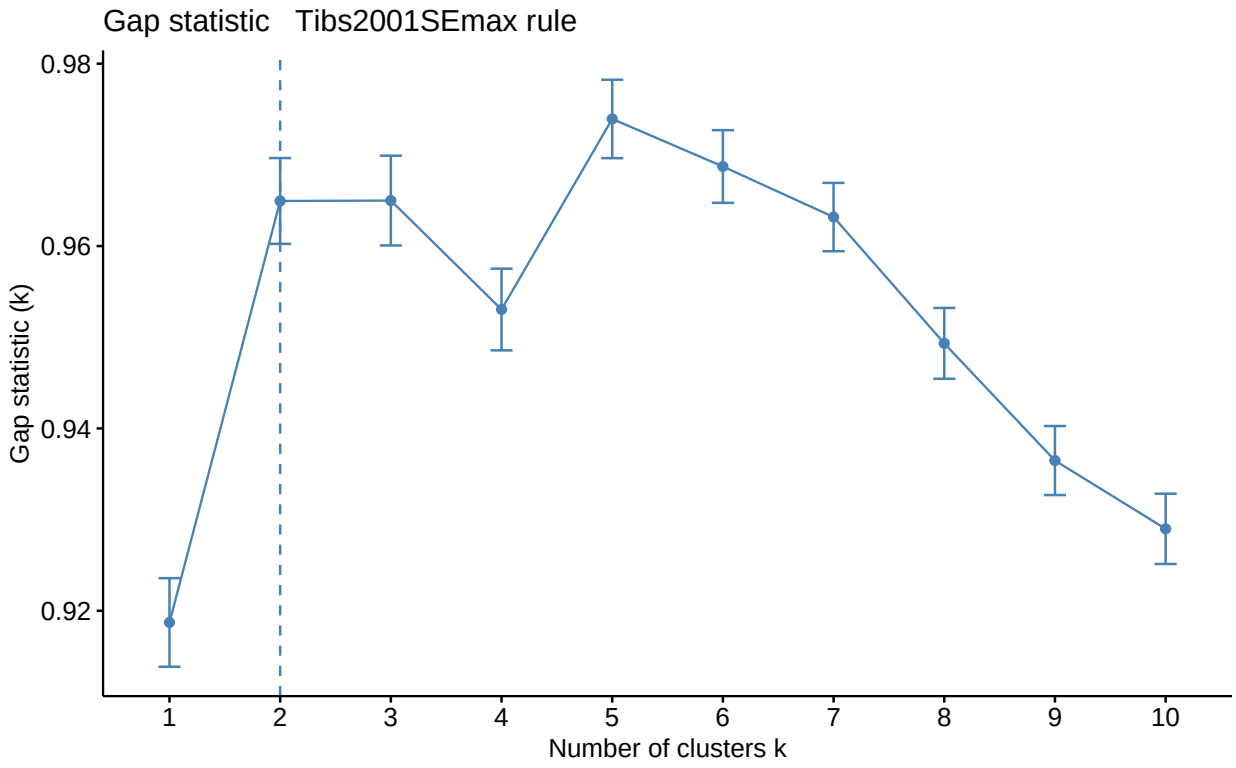
Compares the within-cluster dispersion in the actual data against the within-cluster dispersion under a uniform-noise reference distribution. The **SEmax rule** picks the smallest `k` such that no larger `k` is more than 1 standard error better biased toward parsimony, which is exactly what we want for an interpretable customer segmentation.

```r
set.seed(1)
gap <- clusGap(Xpc, FUN = kmeans, nstart = 25, K.max = 10, B = 50)
print(gap, method = "Tibs2001SEmax")
```

```
## Clustering Gap statistic ["clusGap"] from call:
```

```
## clusGap(x = Xpc, FUNcluster = kmeans, K.max = 10, B = 50, nstart = 25)
## B=50 simulated reference sets, k = 1..10; spaceH0="scaledPCA"
##  --> Number of clusters (method 'Tibs2001SEmax', SE.factor=1): 2
##           logW   E.logW       gap      SE.sim
##  [1,] 8.091630 9.010351 0.9187206 0.004859378
##  [2,] 7.802895 8.767839 0.9649446 0.004704974
##  [3,] 7.695770 8.660760 0.9649899 0.004923846
##  [4,] 7.610874 8.563917 0.9530425 0.004478560
##  [5,] 7.517562 8.491505 0.9739428 0.004302053
##  [6,] 7.453216 8.421940 0.9687238 0.003985563
##  [7,] 7.402958 8.366139 0.9631807 0.003747482
##  [8,] 7.360622 8.309951 0.9493295 0.003884245
##  [9,] 7.327054 8.263533 0.9364784 0.003784421
## [10,] 7.288859 8.217848 0.9289888 0.003854960
```

```
fviz_gap_stat(gap) +
  labs(title = "Gap statistic   Tibs2001SEmax rule")
```



3d. Decision: k = 3, with k = 2 as the parsimonious benchmark

Diagnostics summary:

- The silhouette criterion favors `k = 2`, meaning the two-cluster solution gives the cleanest average geometric separation.
- The gap statistic using the Tibs2001SEmax rule also selects `k = 2`, favoring the most parsimonious structure.
- The elbow plot is gradual and does not provide a decisive visual kink.

- However, `k = 3` still gives a meaningful and stable segmentation when evaluated against the downstream business objective: turning clusters into interpretable customer profiles.

We therefore select **`k = 3`** as the primary profiling solution, while treating **`k = 2`** as the statistically parsimonious benchmark.

This is a deliberate modeling choice. The purpose of this project is not only to find the minimum number of geometrically separated groups, but to produce customer segments that can be interpreted and acted on in `04_profiles.Rmd`. The `k = 2` solution is cleaner statistically, but it collapses the customer base into a broad high-value versus low-value split. That split is useful as a benchmark, but it is too coarse for the profiling stage because it removes the middle customer group that appears in the original-scale features.

The `k = 3` solution gives a more useful structure:

1. a high-value, frequent customer group;
2. a middle/engaged retail group;
3. a lower-value or lapsed/one-time group.

This three-segment structure is more actionable than a two-way split and remains defensible because the bootstrap stability results in §6 show that the clusters are reasonably stable under resampling. Therefore, the final choice of `k = 3` is best described as an **interpretability-driven segmentation choice**, supported by stability analysis, with `k = 2` retained as the simpler statistical benchmark.

```
set.seed(1)
k_star <- 3
km_fit <- kmeans(Xpc, centers = k_star, nstart = 50, iter.max = 100)

km_fit$size
```

```
## [1] 1804 1396 1134
```

```
round(km_fit$centers, 2)
```

```
##     PC1   PC2   PC3
## 1 -0.38 -0.59  0.20
## 2  2.22  0.31 -0.07
## 3 -2.14  0.56 -0.24
```

```
mean(silhouette(km_fit$cluster, d)[, "sil_width"])
```

```
## [1] 0.2692661
```

```
set.seed(1)
km_fit_k3 <- kmeans(Xpc, centers = 3, nstart = 50, iter.max = 100)
km_fit_k3$size
```

```
## [1] 1804 1396 1134
```

```
round(km_fit_k3$centers, 2)
```

```
##     PC1   PC2   PC3
## 1 -0.38 -0.59  0.20
## 2  2.22  0.31 -0.07
## 3 -2.14  0.56 -0.24
```

```r
mean(silhouette(km_fit_k3$cluster, d)[, "sil_width"])
```

```
## [1] 0.2692661
```

4. Sensitivity scaling the PC scores

K-means on raw PC scores weights PC1 more heavily because PC1 has the largest variance by construction. Scaling each PC to unit variance gives every PC equal say in the distance metric. We check whether the segmentation is robust to this choice using the **adjusted Rand index** (`mclust::adjustedRandIndex`): aRI = 1 means identical labels (up to relabelling); aRI = 0 means agreement no better than chance.

```r
set.seed(1)
km_raw    <- kmeans(Xpc,        centers = k_star, nstart = 50, iter.max = 100)
km_scaled <- kmeans(scale(Xpc), centers = k_star, nstart = 50, iter.max = 100)

aRI_scale <- adjustedRandIndex(km_raw$cluster, km_scaled$cluster)
list(adjusted_rand_raw_vs_scaled = round(aRI_scale, 3))
```

```
## $adjusted_rand_raw_vs_scaled
## [1] 0.327
```

This check tests whether the selected two-cluster solution depends heavily on using the raw PC scores. Because raw PC scores preserve the variance ordering from PCA, PC1 receives more weight in the Euclidean distance calculation. That is appropriate here because the goal is to cluster customers along the dominant behavioral directions discovered by PCA. If the adjusted Rand index is low, the report should state that the two-cluster solution is conditional on preserving the PCA variance structure rather than giving every PC equal weight.

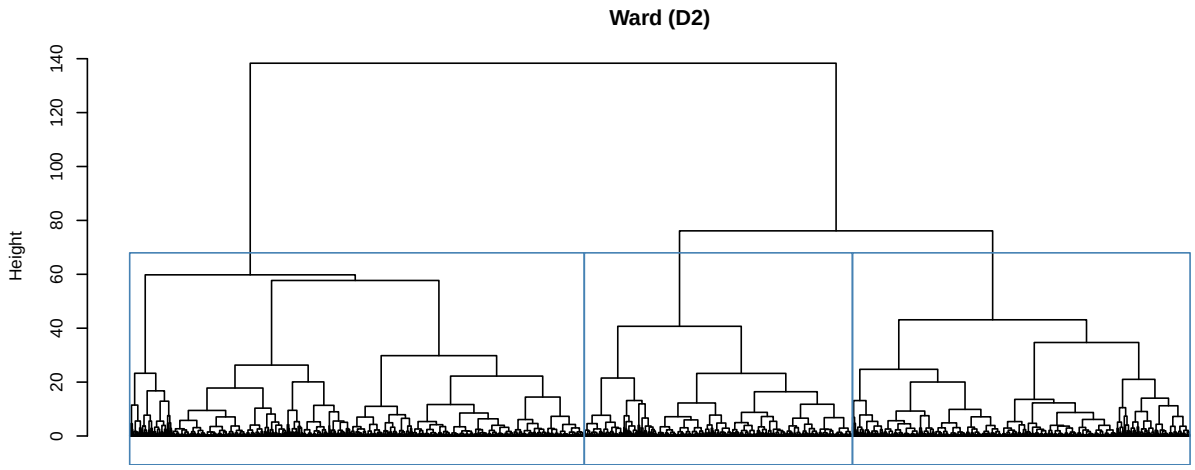
5. Hierarchical clustering

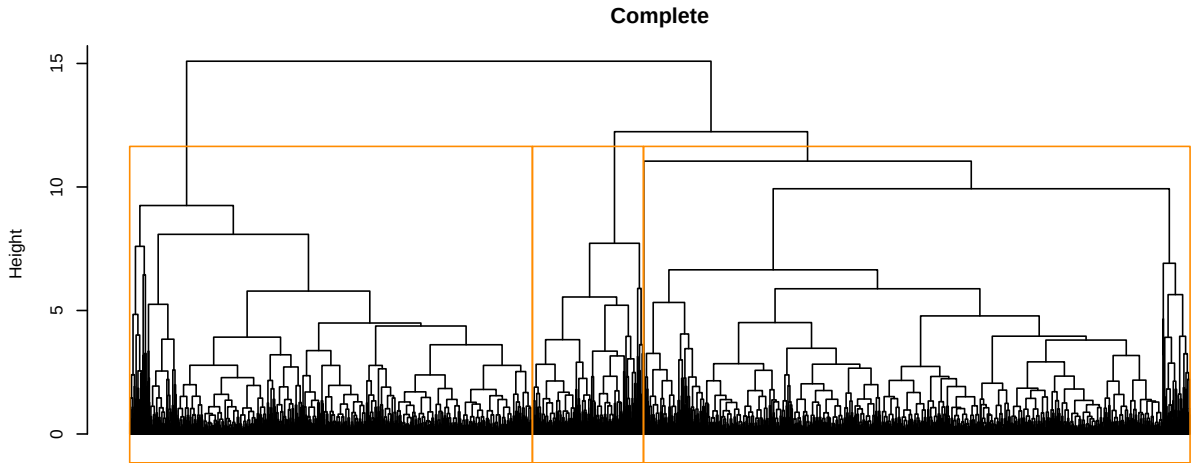
Three linkages worth comparing:

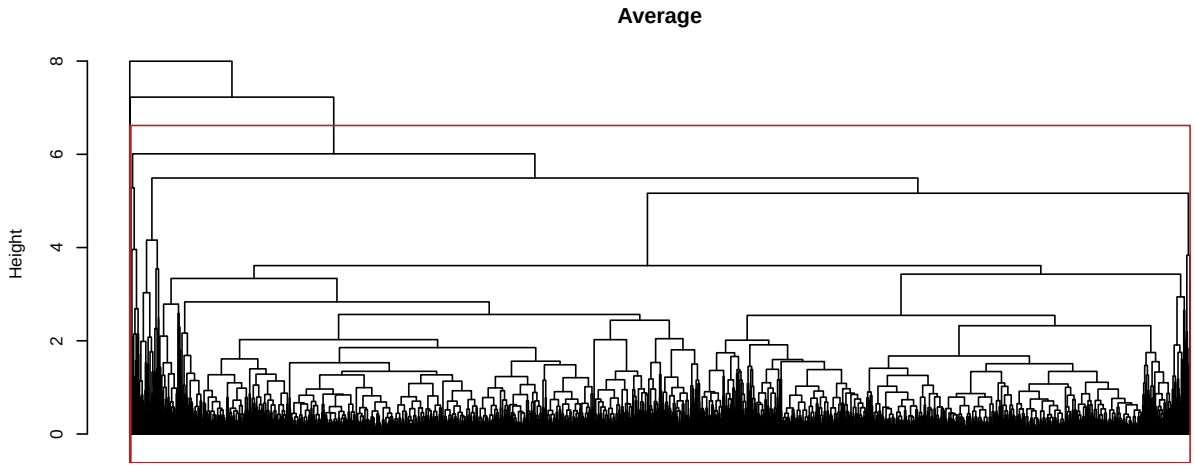
- **Ward (`ward.D2`)** minimises within-cluster variance, philosophically aligned with k-means.
- **Complete** ISLR's lab default; compact, similar-diameter clusters.
- **Average** friendlier to elongated / unequal-density clusters; here as a robustness check.

```r
hc_ward     <- hclust(d, method = "ward.D2")
hc_complete <- hclust(d, method = "complete")
hc_average  <- hclust(d, method = "average")
```

```r
par(mfrow = c(3, 1), mar = c(2, 4, 3, 1))
plot(hc_ward,     labels = FALSE, hang = -1, main = "Ward (D2)",
     xlab = "", sub = "")
rect.hclust(hc_ward,     k = k_star, border = "steelblue")
plot(hc_complete, labels = FALSE, hang = -1, main = "Complete",
     xlab = "", sub = "")
rect.hclust(hc_complete, k = k_star, border = "darkorange")
plot(hc_average,  labels = FALSE, hang = -1, main = "Average",
     xlab = "", sub = "")
rect.hclust(hc_average,  k = k_star, border = "firebrick")
```








```
par(mfrow = c(1, 1))
```

```r
hc_cluster <- cutree(hc_ward, k = k_star)

cross_tab <- table(kmeans = km_fit$cluster, ward = hc_cluster)
cross_tab
```

```
##       ward
## kmeans    1    2    3
##      1  934    9  861
##      2  307 1087    2
##      3  139    0  995
```

```r
aRI_km_hc <- adjustedRandIndex(km_fit$cluster, hc_cluster)
list(adjusted_rand_kmeans_vs_ward = round(aRI_km_hc, 3))
```

```
## $adjusted_rand_kmeans_vs_ward
## [1] 0.383
```

```r
mean(silhouette(hc_cluster, d)[, "sil_width"])
```

```
## [1] 0.2275873
```

Interpretation The cross-tab between k-means and Ward is not perfectly block-diagonal at `k = 3`, so the two algorithms do not assign every customer to the same segment. This is expected because k-means directly optimizes centroid-based within-cluster SSE, while Ward builds the hierarchy greedily by merging clusters that minimally increase within-cluster variance.

The adjusted Rand index quantifies this agreement. A value close to 1 would mean the two methods recover essentially the same partition up to relabelling, while a value close to 0 would mean agreement no better than chance. Here, the agreement is moderate rather than perfect. That means the broad segment structure is present, but customers near the boundaries can move across methods.

We carry the k-means labels forward because k-means is the method used for the formal k-selection diagnostics, its centroids are directly interpretable in PC space, and the bootstrap stability check in §6 is defined for the k-means solution.

```r
prop.table(cross_tab, margin = 1) |>
  round(3) |>
  knitr::kable(caption = paste("Row-normalised overlap: each row is a",
                               "k-means cluster, columns are Ward clusters"))
```

Table 1: Row-normalised overlap: each row is a k-means cluster, columns are Ward clusters

1	2	3
0.518	0.005	0.477
0.220	0.779	0.001
0.123	0.000	0.877

Because cluster labels are arbitrary, the diagonal should not be interpreted mechanically. Instead, each row should be read by its largest entry: this shows which Ward cluster contains most of the customers from a given k-means cluster. If each k-means cluster has one dominant Ward match, the two methods are recovering a similar segmentation; if a row is split across multiple Ward clusters, that k-means segment has a fuzzier boundary.

6. Stability bootstrap Jaccard via `clusterboot`

We use Hennig's `fpc::clusterboot` with non-parametric bootstrap resampling. For each cluster, the **mean Jaccard similarity** across bootstrap replicates measures how often the "same" customers re-cluster together when the input is perturbed. Conventional reading (Hennig 2007):

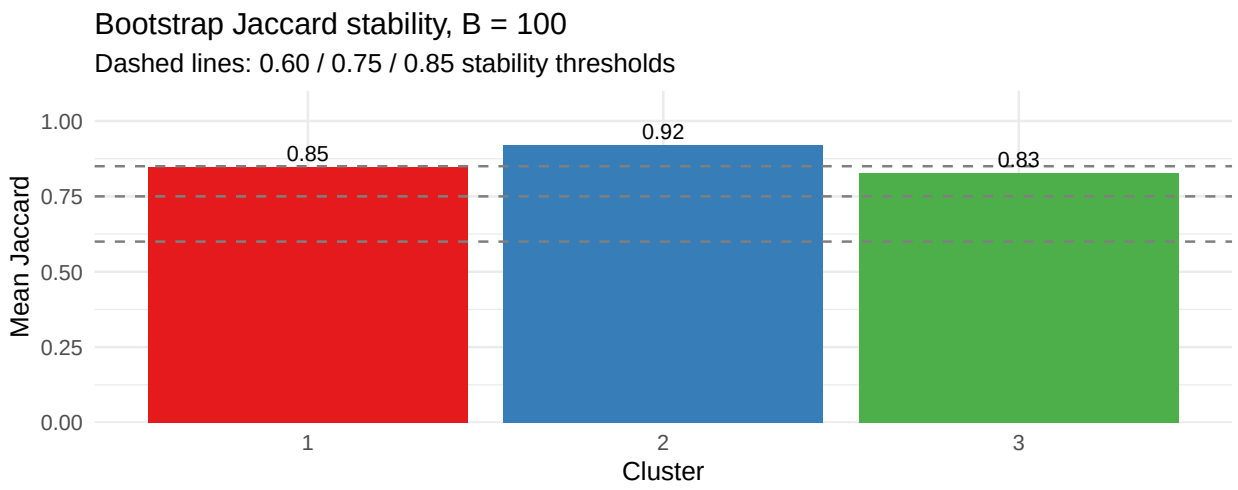
- mean Jaccard ≥ 0.85 highly stable
- 0.75–0.85 stable, the cluster is a real pattern
- 0.60–0.75 measures a pattern, but the boundary is fuzzy
- < 0.60 should not be interpreted as a real cluster

```
set.seed(1)
boot <- fpc::clusterboot(
  data       = Xpc,
  B          = 100,
  bootmethod = "boot",
  clustermethod = kmeansCBI,
  krange     = k_star,
  seed       = 1,
  count      = FALSE
)

stability_df <- tibble(
  cluster      = seq_len(k_star),
  mean_jaccard = round(boot$bootmean, 3),
  dissolved    = boot$bootbrd,        # times Jaccard < 0.5
  recovered    = boot$bootrecover     # times Jaccard >= 0.75
)
stability_df
```

```
## # A tibble: 3 x 4
##   cluster mean_jaccard dissolved recovered
##     <int>        <dbl>     <int>     <int>
## 1       1        0.848         0        78
## 2       2        0.921         0        98
## 3       3        0.826        12        78
```

```
ggplot(stability_df,
       aes(factor(cluster), mean_jaccard, fill = factor(cluster))) +
  geom_col() +
  geom_hline(yintercept = c(0.6, 0.75, 0.85),
             linetype = "dashed", colour = "grey50") +
  geom_text(aes(label = sprintf("%.2f", mean_jaccard)),
            vjust = -0.4, size = 3.5) +
  scale_y_continuous(limits = c(0, 1), expand = expansion(mult = c(0, 0.1))) +
  scale_fill_brewer(palette = "Set1") +
  labs(title = paste("Bootstrap Jaccard stability,", "B = 100"),
       subtitle = "Dashed lines: 0.60 / 0.75 / 0.85 stability thresholds",
       x = "Cluster", y = "Mean Jaccard") +
  theme(legend.position = "none")
```


Bootstrap Jaccard stability, B = 100
Dashed lines: 0.60 / 0.75 / 0.85 stability thresholds
Mean Jaccard
1.00
0.75
0.50
0.25
0.00
0.85
0.92
0.83
1
2
3
Cluster